

Mid-term coding problem set (75 minutes)

Students can use laptop to write your code.

Write your answer to your exam papers.

- Students are not allowed to use phones, web browsers, or social applications (such as Messenger, Zalo, etc.) for any activities during the exam.
- Mobile phones need to be put in your bag, not elsewhere.

Problem 1: Given an array of size $N-1$ such that it only contains distinct integers in the range of 1 to N . Find the missing element. (4 points)

class Solution:

```
def missingNumber(self,array,n):  
    # code here
```

Constraints:

$1 \leq N \leq 10^6$

$1 \leq A[i] \leq 10^6$

A) Design and implement an algorithm with the complexity $O(n^2)$

B) Design and implement an algorithm with the complexity $O(n)$

Problem 2: Design a queue that supports push and pop operations in the front, middle, and back (6 points).

Implement the FrontMiddleBack class:

- *FrontMiddleBack()* Initializes the queue.
- *void pushFront(int val)* Adds val to the front of the queue.
- *void pushMiddle(int val)* Adds val to the middle of the queue.
- *void pushBack(int val)* Adds val to the back of the queue.
- *int popFront()* Removes the front element of the queue and returns it. If the queue is empty, return -1.
- *int popMiddle()* Removes the middle element of the queue and returns it. If the queue is empty, return -1.
- *int popBack()* Removes the back element of the queue and returns it. If the queue is empty, return -1.

Notice that when there are two middle position choices, the operation is performed on the frontmost middle position choice. For example:

Pushing 6 into the middle of [1, 2, 3, 4, 5] results in [1, 2, 6, 3, 4, 5].

Popping the middle from [1, 2, 3, 4, 5, 6] returns 3 and results in [1, 2, 4, 5, 6].

Example 1:

Input:

```
["FrontMiddleBackQueue", "pushFront", "pushBack", "pushMiddle", "pushMiddle", "popFront",  
"popMiddle", "popMiddle", "popBack", "popFront"]  
[[], [1], [2], [3], [4], [], [], [], [], []]
```

Output:

```
[null, null, null, null, null, 1, 3, 4, 2, -1]
```

Explanation:

```

FrontMiddleBackQueue q = new FrontMiddleBackQueue();
q.pushFront(1); // [1]
q.pushBack(2); // [1, 2]
q.pushMiddle(3); // [1, 3, 2]
q.pushMiddle(4); // [1, 4, 3, 2]
q.popFront(); // return 1 -> [4, 3, 2]
q.popMiddle(); // return 3 -> [4, 2]
q.popMiddle(); // return 4 -> [2]
q.popBack(); // return 2 -> []
q.popFront(); // return -1 -> [] (The queue is empty)

```

Constraints:

1 <= val <= 10⁹

Template for the problem:

```
class FrontMiddleBackQueue:
```

```
    def __init__(self):
```

```
    def pushFront(self, val: int) -> None:
```

```
    def pushMiddle(self, val: int) -> None:
```

```
    def pushBack(self, val: int) -> None:
```

```
    def popFront(self) -> int:
```

```
    def popMiddle(self) -> int:
```

```
    def popBack(self) -> int:
```

```

# Your FrontMiddleBackQueue object will be instantiated and called as such:
# obj = FrontMiddleBackQueue()
# obj.pushFront(val)
# obj.pushMiddle(val)
# obj.pushBack(val)
# param_4 = obj.popFront()
# param_5 = obj.popMiddle()
# param_6 = obj.popBack()

```

- A) Justify the design of your data structure;
- B) Implement your data structure
- C) Explain and evaluate the complexity of each operation of your data structures;
- D) Students do not need to implement all the methods

Marking Scheme:

For operations with a complexity of O(1), full marks will be awarded.

For operations with a complexity of O(n), 80% of the full marks will be awarded.

For all other complexities, 50% of the full marks will be awarded for the operation.